Aghalya Narayanan aghalya2
Charan Williams charanw2
Ganesh Saranga saranga2
Jingqia Liu jl126
Keegan Miller kmill34
Nikhil Ajit najit2
Noah Denefe ndenefe2
Zian Xiang zianx2

4th Credit Hour Assignment: LLM Agents

Dec 10th, 2025

Performance

Subset	Resolved Count	Resolved ID
verified-40	18	"django__django-11095", "django__django-11532", "django__django-11880", "django__django-13089", "django__django-13363", "django__django-13417", "django__django-14007", "django__django-14017", "django__django-14855", "django__django-15103", "django__django-15315", "django__django-16136", “matplotlib__matplotlib-2533", "pydata__xarray-6461", "pytest-dev__pytest-5809", "pytest-dev__pytest-7982", "sympy__sympy-17139", "sympy__sympy-19346"
verified-45-extra	Not attempted	Not attempted
verified-50-extra	Not attempted	Not attempted

Design

Improved Read and Write Tool

- Why do you think it is useful?

  The read and write tool gives the agent more flexible “hands” to interact with code and files. Before this tool, the agent could only modify code indirectly via shell commands like “sed” or “awk”, requiring it to rely solely on BASH scripting, which can be inefficient. The Read and Write tool instead uses a structured JSON format that is easier to interpret and less error-prone. In addition, it could help prevent potentially dangerous command line operations by safely handing inputs (for example, through shlex.quote() in Python).

  With improvements, the read and write tool can now accept and return a specified line range, further empowering the agent to more precisely reveal the information it needs and make targeted edits to files.

  Furthermore, the read tool has been improved to include metadata about the file being read, including the total number of lines and file size in bytes, to inform its decisions.

- Are there alternatives, and why are they not chosen?

  Yes. For example, alternatives such as SandBoxed Editors can also be used to debug or edit files within a virtual environment. However, it requires significantly more memory to maintain the sandbox. SWE-Agent utilizes Docker containers for only specific parts of the workflow execution, keeping the process efficient. Additionally, a virtual environment may not fully replicate the real system environment and is temporary, making synchronization more challenging. In contrast, the read and write tool, integrated into the SWE-agent, enables more direct, efficient, and synchronized interaction with project files.

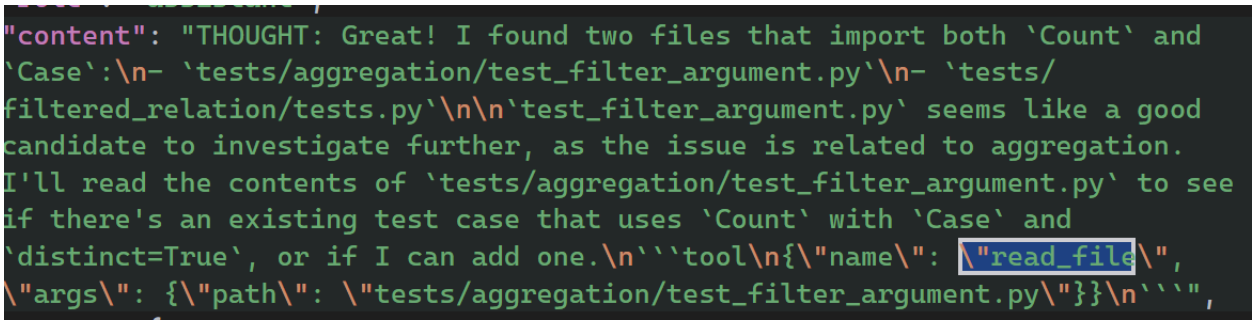
- How did you implement it?

  We expanded upon the provided basic read-and-write tools. Read takes the file path to read, along with optional 1-indexed, inclusive start and stop line numbers. Reading is done using a ‘sed’ command, with fallback to return the full file and avoid truncation. Then the lines are converted to numbered lines using the line separator “\n” to count, and metadata is calculated. The selected content is returned along with a return code.

The write tool takes the file path and the content to write as arguments. It encodes the content using base64 to prevent issues with special characters. It uses ‘mkdir’ to ensure the file exists and then prints to the file. It returns the path to the content written and a return code.

- Was it actually useful?

Both the read and write tools are reused internally throughout other tool implementations. The agent makes extensive use of this tool. An example of the agent reading a file is shown below:



ApplyDiff Tool

- Why do you think it is useful?

This tool enables the agent to make code edits using standard unified-diff patches. This can reduce errors and make the model more precise and efficient in its edits.

- Are there alternatives, and why are they not chosen?

An alternative is to use the existing write tool directly. While this would be an improvement over using the sed command, the ApplyDiff tool handles merges and conflicts more effectively.

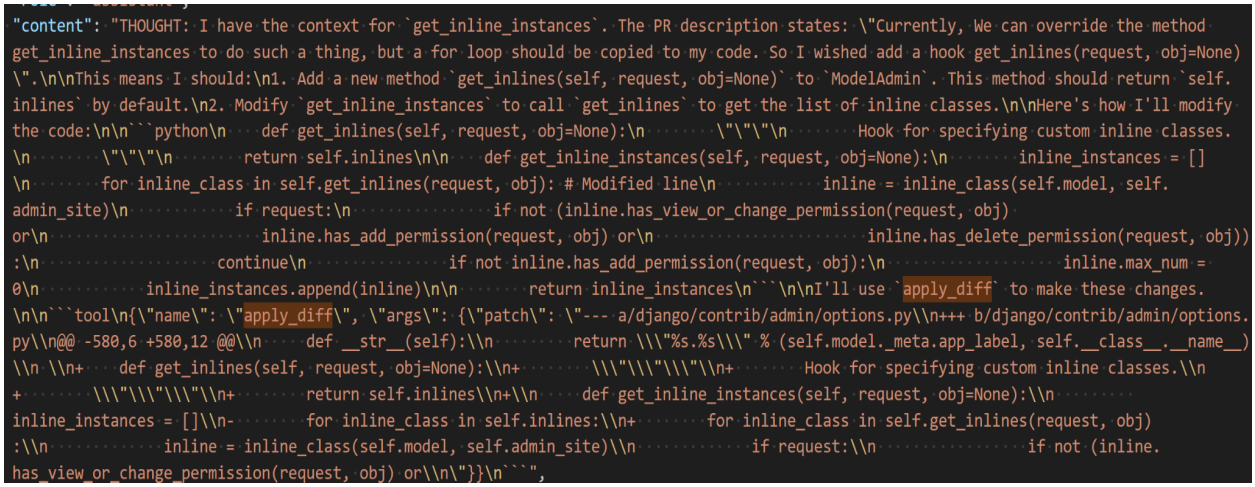
- How did you implement it?

The tool accepts a string containing the diff patch with an optional strip value. The strip is used to remove file paths from the diff that may not actually exist in the working directory. It performs normalization by replacing ‘\r’ line endings with ‘\n\. It also removes markdown backticks for code blocks and then performs basic validation to ensure the content is actually in git diff format.

It then writes the patch to a temporary file using the write tool, and performs a dry run using the patch ‘--dry-run’ flag. If the dry run succeeds, it then applies the patch using ‘patch’. It returns simple output messages and return codes to inform the agent of the outcome from applying the patch.

- Was it actually useful?

As shown in the screenshot below, the agent used the apply-diff tool to apply its patch and confirm its changes:



GitDiff Tool

- Why do you think it is useful?

  GitDiff enables the agent to inspect uncommitted modifications in the working directory. This could help prevent accidental edits and help the agent make informed decisions on its next action based on the current state of the code.

- Are there alternatives, and why are they not chosen?

  An alternative would be to use the Read tool; however, this requires the agent to read whole files, which could increase the risk of hallucinations. The model could also use the search tool; however, this still doesn't convey the advantage of providing the standardized git diff.

- How did you implement it?

  This is a straightforward tool that executes git diff inside the sandbox. It takes no arguments and returns the output from git diff and a return code. If there is no output from git diff, it returns a simple message of "No changes detected."

- Was it actually useful?

  The agent used this tool to confirm that its changes were made properly:



LintFile Tool

- Why do you think it is useful?

This tool enables the agent to catch syntax errors without running Python files. It can also help the agent catch syntax errors without applying patches. By performing a syntax check, the agent can catch its own mistakes sooner and avoid wasting iterations on broken code that a linter would catch.

- Are there alternatives, and why are they not chosen?
  An alternative is to run the file directly; however, this could trigger other errors that may have been caused by syntax issues and lead to unnecessary iterations spent on troubleshooting.

- How did you implement it?
  This tool wraps the Python py_compile command and accepts a path to the file to lint as the argument. It outputs a simple message of “Syntax error detected” with the result of the command, along with a return code.

- Was it actually useful?
  Unfortunately, the LLM did not utilize the lint file tool.

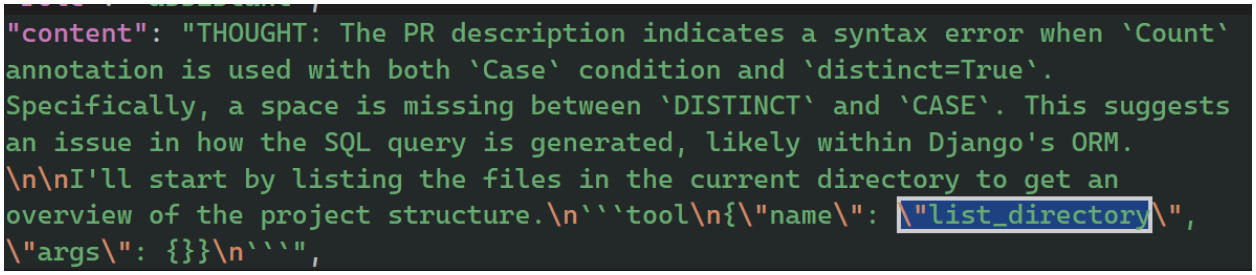
ListDirectory Tool

- Why do you think it is useful?
  Giving the agent the ability to see the contents of a given directory with various file depths can give the AI more context about the file it is debugging. This is especially useful if the project falls into a particular framework or if the file names are descriptive enough to provide further information about what they do.

- Are there alternatives, and why are they not chosen?
  The agent already had access to the ‘ls’ command. This tool extends that functionality by allowing the agent to provide the file tree depth to return within the file structure, and by giving the model a structured way to use the command while guiding when and how to use it properly.

- How did you implement it?
  This tool accepts two arguments: a string representing the path to the list and an integer representing the depth to return. Defaults are set to ‘.’ and 2. This tool wraps the ‘find’ command with the ‘maxdepth’ flag. It directly returns the result of executing the command.

- Was it actually useful?
  As seen below, the AI uses the tool to get an overview of the project structure:



Replace Tool

- Why do you think it is useful?

  The replace tool enables the agent to make simple changes directly to a file. Previously, the agent had no way to efficiently change files other than using the write tool. Advantages from using the replace tool include more precise edits and less risk of mistakes, as generating diff patches creates more opportunities for mistakes and hallucinations.

- Are there alternatives, and why are they not chosen?

  While alternative tools, such as write and apply diff, exist, they have also been incorporated into the agent's toolbelt. I could've written a replace tool that replaced single instances, but in many cases, replace is more global & using such a tool could lead to replacements being skipped. This tool complements the apply diff tool. It is ideal for small, straightforward changes, while the apply diff tool is more suited to larger structural changes.

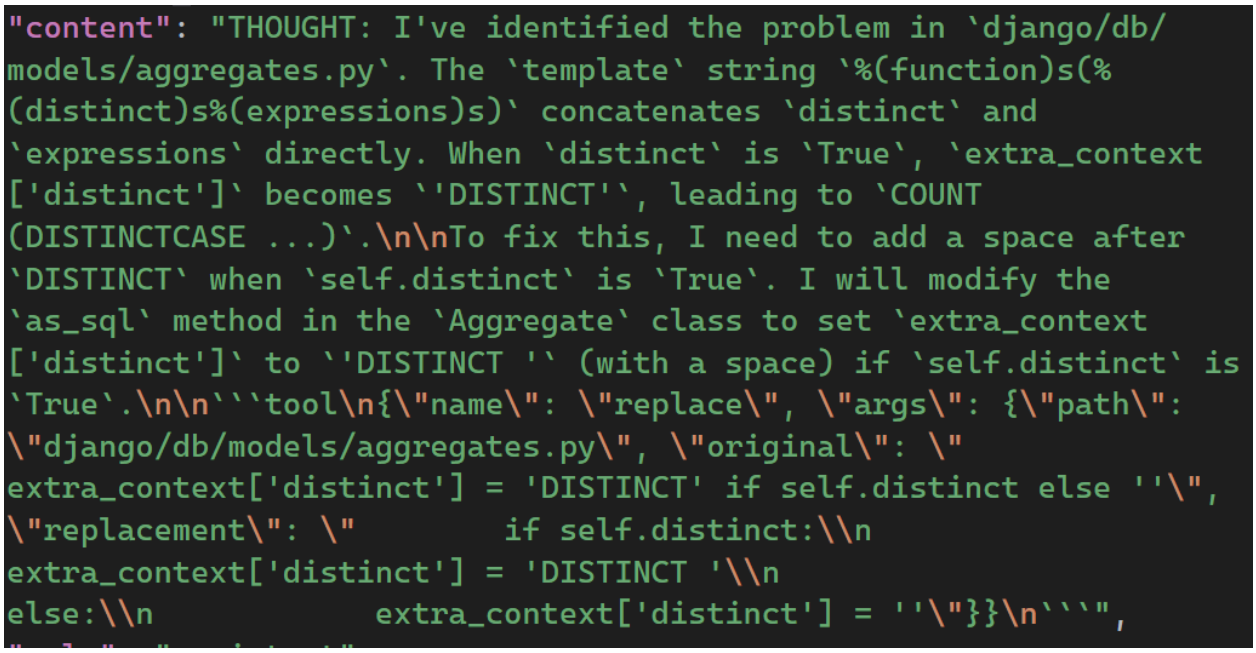
- How did you implement it?

  The Replace tool was initially written using a sed command to globally replace one pattern with another in a given file. This was done so that all instances of the pattern could be replaced within a file simultaneously. It was then rewritten to use the read & write tools. This change was made because the agent already prefers to use the sed command on its own, and replacing only exact matches reduces the risk of overwriting unrelated code.

  It accepts three strings as input: the path to the file, the original value to search for, and the replacement value to insert. It performs validation to ensure all arguments are present, then reads the file using the read tool. It uses text.replace() to perform the replacement, then writes back to the file using the write tool. It returns simple messages confirming the status of the change and a return code.

- Was it actually useful?

  The tool was used by the agent in situations where smaller changes needed to be applied to a file without generating an entire diff. An example of the agent using the replace tool is shown below:



```
"content": "THOUGHT: I've identified the problem in `django/db/
models/aggregates.py`. The `template` string `%(function)s(%
(distinct)s%(expressions)s)` concatenates `distinct` and
`expressions` directly. When `distinct` is `True`, `extra_context
['distinct']` becomes ''DISTINCT'', leading to `COUNT
(DISTINCTCASE ...)`.\n\nTo fix this, I need to add a space after
`DISTINCT` when `self.distinct` is `True`. I will modify the
`as_sql` method in the `Aggregate` class to set `extra_context
['distinct']` to ''DISTINCT '' (with a space) if `self.distinct` is
`True`.\n\n```tool\n{\"name\": \"replace\", \"args\": {\"path\":
\"django/db/models/aggregates.py\", \"original\": \"
extra_context['distinct'] = 'DISTINCT' if self.distinct else ''\",
\"replacement\": \"        if self.distinct:\\n
extra_context['distinct'] = 'DISTINCT '\\n
else:\\n            extra_context['distinct'] = ''\"}}\n```",
```

RunTest Tool

- Why do you think it is useful?

  Giving the AI SWE_Agent the ability to validate its fixes is useful as it can ensure that the fix actually resolved the issue under test. If not, then the SWE_Agent could try further debugging.

- Are there alternatives, and why are they not chosen?

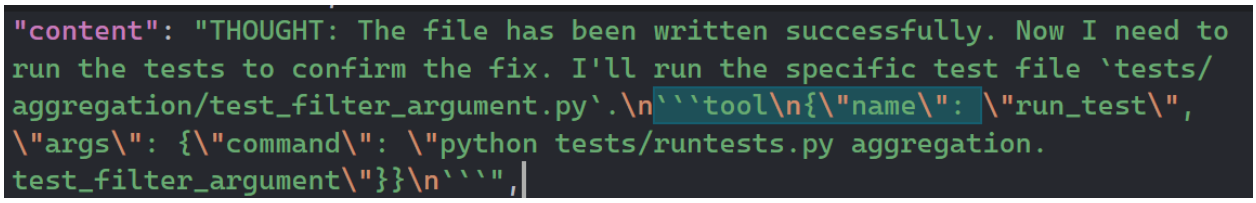
  Without this tool, the model has to use bash/execute to run the files. This tool gives the model a framework for running the test and helps it understand that it needs to test its code. Another alternative is to let the model reason about the patch and determine whether it should work on its own, but this is prone to hallucinations and requires many more tokens.

- How did you implement it?

  This tool is really just a wrapper that allows the agent to run ‘execute’. It takes a string (the command to run) as input and returns the command's output directly.

- Was it actually useful?

  An example of the tool being used to run test files is shown below:



SearchFiles Tool

- Why do you think it is useful?

  This tool is useful because it now enables the agent to search through files and locate desired keywords or phrases. Before this tool was implemented, the agent had no structured way to search through files to retrieve helpful information. Instead of relying on direct commands like grep and find, the agent can now more effectively search through the file directory.

- Are there alternatives, and why are they not chosen?

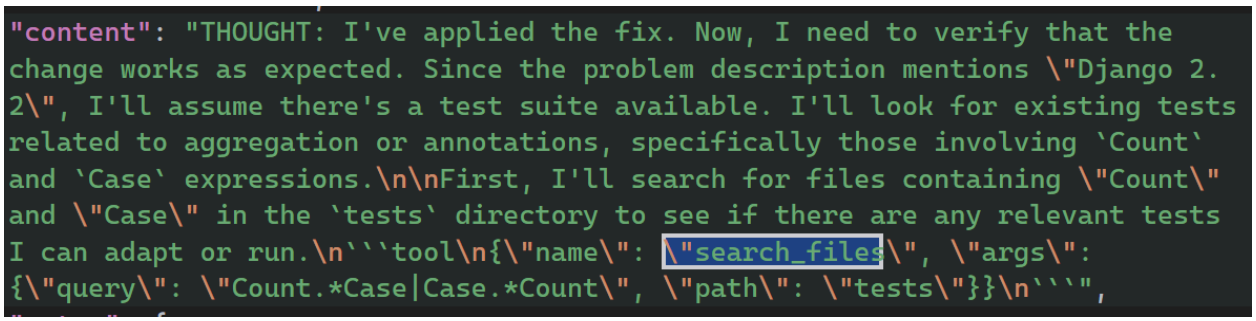
  Several alternatives exist, such as python-native recursive search using os.walk and manual file parsing. However, these approaches are slower and more complex, especially on large repositories. The grep utility, by contrast, is fast, lightweight, and available for all environments. It provides reliable, predictable behavior with minimal overhead.

- How did you implement it?

  The tool wraps the ‘grep’ command, accepting a query to search for and a directory path, both as strings. The path defaults to ‘.’.The use of shlex.quote ensures that supplied input is safely escaped, preventing shell injection, which may lead to workspace corruption. It returns the result of executing the command directly.

- Was it actually useful?

  This tool is frequently used by the agent. An example of the tool being used to search for test files is shown below:



SummarizeFile Tool

- Why do you think it is useful?

  This tool enables the agent to quickly understand a file's overall structure and purpose. Large files can overwhelm the agent if passed directly into context. This helps the agent decide whether a file is relevant before investigating further. This can help accelerate debugging by making it easier for the agent to navigate the codebase and reason about large files while maintaining context.

- Are there alternatives, and why are they not chosen?

  An alternative is the read tool. However, reading an entire large file can exceed context limits and doesn't provide an immediate high-level understanding. The agent would still need to process and summarize the content independently, which is inefficient. This tool guides the agent to preview a file before proceeding within its workflow, to help it maintain state and be more efficient within its context.

- How did you implement it?

  This tool accepts two arguments: a path to the file to summarize, and a maximum length to return, as a string and an integer, respectively. It uses the read tool to read the file, then truncates its contents.

- Was it actually useful?

  Unfortunately, the agent did not utilize the tool during our runs.

ToDoList Tool

- Why do you think it is useful?

  This tool is useful because it gives the agent a long-term memory that otherwise wouldn't have been possible. Out of the box, the agent can't truly remember anything between tool calls if it's pushed outside its context window. This new functionality allows the agent to keep track of its "TODOS" like a software engineer, supporting multi-step reasoning.

- Are there alternatives, and why are they not chosen?

  One alternative is the memory concept that was available as extra credit for this course. We did not opt for this due to time constraints, as we focused on other tools and the main Android application project. This tool should provide some of the benefits with less of a time sink.

- How did you implement it?

  This tool takes two strings and an integer as input. It requires a string representing the action to perform from an enumerated list: ["add", "remove", "list", "clear"], and optionally accepts a string of the description of what to add to the to-do list,

or an integer for the 1-based index of what to remove from the to-do list. The to-do list is saved as a .json file and supports adding, removing, displaying, and clearing the list using .append(), .pop(), enumerate(), or setting the list to an empty string, respectively.

If the list is empty, it displays a message, and it returns a simple message stating the action completed and a return code.

- Was it actually useful?
  Unfortunately, the agent did not utilize this tool during our runs.

Learnings

While we did not pursue the optional last 10 instances from the SWE bench verified-50, we encountered and overcame many challenges while executing the assignment.

During the project, several team members repeatedly hit rate limits from the Gemini API. These were encountered even when using API keys with available credits. We initially assumed it was a quota issue outside of our control. After further investigation, we discovered that the agent was still calling the free public Gemini endpoint because it was being passed in through the command; the model name needed to be changed from gemini/gemini-2.5-flash to vertex_ai/gemini-2.5-flash. Fully setting up the API access required generating a Vertex AI service account, creating a JSON key file, and updating the GOOGLE_APPLICATION_CREDENTIALS path in the MiniSWE config. Once this was corrected, the agent finally stopped rate-limiting and began running more reliably. Changes to Google's API policies and frontend during the semester further complicated this.

Another challenge arose in understanding how to provide the agent with access to the tools. At first it appeared that it never used certain tools, such as the Replace tool. After some troubleshooting, we realized the tool had to be imported into **__init__**.py file within the tools folder.

After fixing imports and improving tool descriptions, the agent began using the newly added tools more consistently during SWE-bench runs. This showed us that proper tool integration and clear descriptions are just as important as the tool's functionality.

A final environment challenge which arose were issues running the SWE bench evaluation tool in the WSL environment. The overhead from running Docker images on WSL on top of Windows led to hang ups, freezes, and corrupted images which required multiple resets of the working environment.